

Computer Organization and Architecture

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Computer Organization and Architecture — One-Liners	3
Number Systems	3
Binary Arithmetic and Complements	3
Data Representation	3
Codes — Gray, BCD, Excess-3, ASCII, EBCDIC	4
Error Detection and Correction	4
Digital Logic — Gates and Boolean Algebra	4
Digital Logic — K-Map and Circuits	4
Digital Logic — Sequential Circuits	5
Instruction Set and Formats	5
Addressing Modes	5
Program Control — PC, Stack, RPN	5
CPU Organization and Registers	6
Control Unit and Microprogramming	6
Pipelining and Hazards	6
RISC vs CISC and Flynn's Classification	6
Memory Hierarchy and Types	7
Cache Memory	7
Virtual Memory and Management	7
Input-Output Organization	7

Computer Organization and Architecture — One-Liners

High-density, double-layered rapid-revision facts for the **IBPS SO (IT Officer)** exam. Every line pairs a structural metric (how many / which / what size / what value) with the descriptive answer.

Number Systems

1. There are **4** common number systems: **binary (base 2)**, **octal (base 8)**, **decimal (base 10)**, and **hexadecimal (base 16)**.
2. Binary uses **2** digits — **0 and 1** — while hexadecimal uses **16** symbols — **0–9 and A–F**.
3. The base of a number system equals the **count of distinct digits** it uses, e.g., octal has **8** digits (**0–7**).
4. Binary-to-octal conversion groups bits in **3s**, because **one octal digit = 3 bits**.
5. Binary-to-hexadecimal conversion groups bits in **4s**, because **one hex digit = 4 bits (a nibble)**.
6. One byte holds **2** hexadecimal digits, since a byte is **8 bits** and each hex digit is **4 bits**.
7. To convert a decimal integer to any base you use **repeated division**, reading remainders **bottom-to-top**.
8. To convert a decimal fraction to any base you use **repeated multiplication**, reading carries **top-to-bottom**.
9. The hex value **A** equals decimal **10**, and **F** equals decimal **15**.
10. The positional weight of a digit is **base raised to its position**, with the integer side starting at position **0**.
11. The largest single digit in base r is $r - 1$, so the largest binary digit is **1** and the largest octal digit is **7**.
12. Octal 745 expands using weights **64, 8, 1** (i.e., $8^2, 8^1, 8^0$).

Binary Arithmetic and Complements

1. There are **2** complements for base r : the **$(r-1)$'s complement (diminished radix)** and the **r 's complement**.
2. The binary complements are **2**: the **1's complement (flip all bits)** and the **2's complement ($1's + 1$)**.
3. The decimal complements are **2**: the **9's complement** and the **10's complement ($9's + 1$)**.
4. The fast 2's-complement trick scans from the **LSB**, copying up to the **first 1** then **inverting** the rest.
5. In binary addition, **$1 + 1 = 10$** , meaning write **0** and carry **1**.
6. Binary subtraction borrows **2** from the next higher position, unlike decimal which borrows **10**.
7. In 2's-complement subtraction an **end carry is discarded**, signalling a **positive** result.
8. In 1's-complement subtraction the end carry is **added back**, called the **end-around carry**.
9. Booth's algorithm performs **signed multiplication** efficiently using **2's-complement** encoding.
10. Overflow needs **same-sign operands**, detected when **carry-in to MSB $\neq$ carry-out of MSB**.
11. Adding **opposite-sign** numbers can **never** cause overflow in 2's-complement arithmetic.
12. For n bits, 2's-complement range is **$-2^{(n-1)}$ to $+2^{(n-1)} - 1$** , e.g., 8-bit is **-128 to $+127$** .

Data Representation

1. There are **3** common signed-integer schemes: **signed-magnitude**, **1's complement**, and **2's complement**.
2. Exactly **1** scheme has a unique zero — **2's complement** — while signed-magnitude and 1's complement have **two zeros (± 0)**.
3. The MSB in signed representations is the **sign bit**: **0 = positive**, **1 = negative**.
4. Signed-magnitude range for n bits is **$\pm(2^{(n-1)} - 1)$** , with **2** representations of zero.
5. Fixed-point representation keeps the **radix point fixed**, giving **exact integers** but **limited range**.
6. A floating-point number has **3** fields: **sign (S)**, **exponent (E)**, and **mantissa/fraction (M)**.
7. IEEE-754 single precision is **32 bits**: **1** sign, **8** exponent, **23** mantissa.
8. IEEE-754 double precision is **64 bits**: **1** sign, **11** exponent, **52** mantissa.
9. The single-precision bias is **127** and the double-precision bias is **1023**, both equal to **$2^{(e-1)} - 1$** .
10. The floating-point value formula is **$(-1)^S \times 1.M \times 2^{(E - \text{bias})}$** , with the leading **1 implicit (hidden bit)**.
11. IEEE-754 encodes **$\pm$ infinity** when **$E = \text{all 1s}$ and $M = 0$** , and **NaN** when **$E = \text{all 1s}$ and $M \neq 0$** .
12. IEEE-754 encodes **$\pm$ zero** when **$E = 0$ and $M = 0$** , and **denormalized** numbers when **$E = 0$ and $M \neq 0$** .
13. Normalization places exactly **one nonzero digit (a 1)** left of the radix point, giving form **1.xxxx**.

- The exponent is stored **biased** so it is added when **storing** and subtracted when **reading**.

Codes — Gray, BCD, Excess-3, ASCII, EBCDIC

- Codes are of **2** weight-types: **weighted (BCD/8421)** and **unweighted (Gray, Excess-3)**.
- Gray code changes exactly **1** bit between successive values, making it a **unit-distance / reflected** code.
- Binary-to-Gray keeps the **MSB same** and sets each next bit as $G_i = B_i \text{ XOR } B_{(i+1)}$.
- Gray-to-binary keeps the **MSB same** and sets each next bit as $B_i = B_{(i+1)} \text{ XOR } G_i$.
- Gray code is mainly used in **shaft/position encoders** and **K-Maps** to reduce switching errors.
- BCD (8421) encodes each decimal digit in **4 bits** using weights **8-4-2-1**.
- BCD has **6** invalid combinations — the patterns **1010 to 1111** — leaving only **10** valid codes.
- BCD addition adds **6 (0110)** as correction whenever a digit-sum **exceeds 9** or generates a carry.
- Excess-3 code equals **BCD + 3 (0011)** and is both **unweighted** and **self-complementing**.
- Self-complementing codes give the **9's complement** by simple **bit inversion**, e.g., **Excess-3, 2421, 84-2-1**.
- ASCII is a **7-bit** code representing **128** characters, extendable to **8 bits / 256** characters.
- EBCDIC is an **8-bit** code representing **256** characters, used on **IBM mainframes**.
- In ASCII, **'A' = 65**, **'a' = 97**, and **'0' = 48**, so case differs by **32**.
- Weighted vs unweighted: **BCD is weighted (8421)** whereas **Gray and Excess-3 are unweighted**.

Error Detection and Correction

- A parity bit adds **1** extra bit and detects an **odd number (single-bit)** of errors.
- There are **2** parity schemes: **even parity (total 1s even)** and **odd parity (total 1s odd)**.
- Parity can **detect** but **cannot correct**, and it misses an **even number** of errors.
- The **odd function** is the **XOR** gate, output **1** when an **odd** number of inputs are 1.
- The **even function** is the **XNOR** gate, output **1** when an **even** number of inputs are 1.
- Even-parity generation uses **XOR**, while odd-parity generation uses **XNOR**.
- Hamming code corrects a **single-bit** error and needs **p** parity bits where $2^p \geq m + p + 1$.
- Hamming parity bits sit at **power-of-2** positions: **1, 2, 4, 8, ...**
- Hamming distance is the **count of differing bit positions** between two codewords.
- To **detect d** errors you need minimum distance **d + 1**; to **correct d** errors you need **2d + 1**.

Digital Logic — Gates and Boolean Algebra

- There are **7** basic logic gates: **AND, OR, NOT, NAND, NOR, XOR, XNOR**.
- The **2** universal gates are **NAND and NOR**, each able to build any logic function alone.
- The XOR gate outputs **1** when its inputs **differ**, acting as an **inequality** detector.
- The XNOR gate outputs **1** when its inputs are **equal**, acting as a **1-bit comparator**.
- De Morgan's theorems are **2**: $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$.
- The identity laws state $A + 0 = A$ and $A \cdot 1 = A$.
- The complement laws state $A + A' = 1$ and $A \cdot A' = 0$.
- The absorption law states $A + AB = A$ and $A(A + B) = A$.
- The consensus theorem states $AB + A'C + BC = AB + A'C$, dropping the redundant **BC** term.
- A NOT gate (inverter) has **1** input, while basic AND/OR gates have **2 or more** inputs.

Digital Logic — K-Map and Circuits

- A K-Map arranges cells in **Gray-code order** so adjacent cells differ by exactly **1** bit.
- K-Map groups must be **powers of two** — sizes **1, 2, 4, 8, 16** — and larger groups remove more variables.
- Grouping **1s** yields minimal **SOP**, while grouping **0s** yields minimal **POS**.
- Don't-care (X)** terms can be included in groups to produce a **simpler** expression.
- Combinational circuits depend only on **present inputs** and have **no memory**.

6. A half adder gives **Sum = A XOR B** and **Carry = A·B**, adding **2** bits.
7. A full adder gives **Sum = A XOR B XOR Cin**, adding **3** bits.
8. A multiplexer with **n** select lines chooses among **2ⁿ** inputs to drive **1** output.
9. A decoder converts **n** inputs into **2ⁿ** outputs, while an encoder does the reverse (**2ⁿ → n**).
10. A MUX is a **data selector** (many→one); a DEMUX is a **data distributor** (one→many).

Digital Logic — Sequential Circuits

1. Sequential circuits depend on **present inputs and past state**, requiring **memory** and a **clock**.
2. There are **4** flip-flop types: **SR, JK, D, and T**.
3. The **SR** flip-flop has **1** forbidden input combination — **S = R = 1**.
4. The **JK** flip-flop **toggles** when **J = K = 1**, removing SR's invalid state.
5. The **D** flip-flop has **no invalid state**; its output simply **follows D** (delay/data).
6. The **T** flip-flop **toggles** the output whenever **T = 1**.
7. A latch is **level-triggered**, while a flip-flop is **edge-triggered**.
8. Shift registers come in **4** types: **SISO, SIPO, PISO, PIPO**.
9. Counters are **2** kinds: **asynchronous/ripple (slow)** and **synchronous (fast, shared clock)**.
10. An n-bit register stores **n** bits and a modulo-N counter cycles through **N** states.

Instruction Set and Formats

1. A machine instruction has **2** main parts: the **opcode** and one or more **operand/address** fields.
2. Instruction formats are **4** by address count: **zero-, one-, two-, and three-address**.
3. **Zero-address** instructions operate on a **stack** using **PUSH/POP** with implicit operands.
4. **One-address** instructions use an implied **accumulator** for one operand.
5. **Three-address** instructions name **2 sources and 1 destination** explicitly.
6. The instruction cycle has **3** core phases: **fetch, decode, execute** (plus optional interrupt).
7. During fetch the **PC** supplies the address and is then **incremented**.
8. Machine language is **binary (0s and 1s)** and is the **only** language hardware executes directly.

Addressing Modes

1. **Immediate** mode stores the **operand inside the instruction**, needing **0** memory references for the operand.
2. **Direct** mode needs **1** memory reference, with the address field holding the **effective address**.
3. **Indirect** mode needs **2** memory references, since the address field points to the **address of the operand**.
4. **Register** mode is the **fastest**, keeping the operand **inside a CPU register**.
5. **Register-indirect** mode places the **operand's address in a register**.
6. **Indexed** mode computes **EA = base address + index register**, ideal for **arrays**.
7. **Base-register** mode computes **EA = base register + offset**, used for **relocation**.
8. **Relative** mode computes **EA = PC + offset**, enabling **position-independent branches**.
9. **Auto-increment/decrement** modes adjust a register **after/before** use, useful for **stacks and arrays**.

Program Control — PC, Stack, RPN

1. The Program Counter (PC) holds the address of the **next** instruction to fetch.
2. A subroutine **CALL** saves the **return address** (on a stack/link register) and loads the routine address.
3. A subroutine **RETURN** pops the saved address back into the **PC**.
4. A stack is a **LIFO** structure managed by a **stack pointer (SP)** marking the **top**.
5. **PUSH** adds to the **top** of the stack; **POP** removes from the **top**.
6. Reverse Polish Notation (postfix) places the operator **after** operands and needs **no parentheses**.
7. RPN is evaluated naturally by a **stack**, which is why it suits **zero-address** machines.
8. Infix **(A+B)*C** becomes postfix **AB+C***.

CPU Organization and Registers

1. The CPU has **3** core components: the **ALU**, the **Control Unit (CU)**, and the **register set**.
2. The ALU performs **2** classes of operations: **arithmetic** and **logic**.
3. The **PC** holds the next instruction address; the **IR** holds the **current** instruction.
4. The **MAR** holds the **memory address**, while the **MDR/MBR** holds the **data** to/from memory.
5. The **Accumulator (AC)** stores **ALU results** in accumulator-based machines.
6. The **PSW** holds **status flags / condition codes** like Zero, Carry, Sign, Overflow.
7. The ALU sets **4** common flags: **Zero, Carry, Sign, and Overflow**.
8. General-register organization uses **many GPRs** to reduce slow **memory** accesses.
9. A common internal **bus** interconnects the ALU, registers, and CU inside the CPU.
10. The stack pointer (SP) always points to the **top** element of the CPU stack.

Control Unit and Microprogramming

1. There are **2** control-unit designs: **hardwired** and **microprogrammed**.
2. **Hardwired** control is **faster but rigid**, and is typical of **RISC** CPUs.
3. **Microprogrammed** control is **flexible but slower**, and is typical of **CISC** CPUs.
4. A **control word** is a bit pattern where each bit enables one **micro-operation/control signal**.
5. A **microprogram** is a sequence of **microinstructions** stored in **control memory (control ROM)**.
6. A **microinstruction** in turn contains one or more **micro-operations**.
7. Microcode is **2** styles: **horizontal (wide, fast)** and **vertical (narrow, encoded, slower)**.
8. The **Control Address Register (CAR)** sequences the next **microinstruction** address.

Pipelining and Hazards

1. A classic RISC pipeline has **5** stages: **IF, ID, EX, MEM, WB**.
2. Pipelining overlaps instructions like an **assembly line**, improving **throughput**.
3. For **k** stages and **n** instructions, pipelined time is **(k + n - 1)** cycles.
4. Pipeline **speedup** = $nk / (k + n - 1)$, with a **maximum of k**.
5. There are **3** hazard types: **structural, data, and control**.
6. **Structural** hazards arise from **resource conflicts**, e.g., a single memory port.
7. **Data** hazards are **3: RAW, WAR, and WAW**.
8. **RAW (Read After Write)** is the **true dependency**, the most common hazard.
9. **WAR** is the **anti-dependency** and **WAW** is the **output dependency**.
10. **Control** hazards come from **branch/jump** instructions changing the **PC**.
11. **Forwarding/bypassing** resolves most **RAW** hazards without stalling.
12. **Stalling** inserts **bubbles (no-ops)** until a hazard clears.
13. **WAR and WAW** name dependencies are removable by **register renaming**.
14. Control hazards are mitigated by **branch prediction, delayed branch, and flushing**.

RISC vs CISC and Flynn's Classification

1. RISC uses **fixed-length** instructions; CISC uses **variable-length** instructions.
2. RISC mostly executes in **1 cycle**; CISC is **multi-cycle**.
3. RISC accesses memory only via **load/store**; CISC allows **many** memory-operand instructions.
4. RISC uses **hardwired** control and **many** registers; CISC uses **microprogrammed** control and **fewer**.
5. RISC examples include **ARM, MIPS, RISC-V**; CISC examples include **x86, VAX**.
6. Flynn's taxonomy defines **4** classes: **SISD, SIMD, MISD, MIMD**.
7. **SISD = 1 instruction stream, 1 data stream** — a classic uniprocessor.
8. **SIMD = 1 instruction, many data**, used by **GPUs and vector processors**.
9. **MISD = many instructions, 1 data** — the **rarest** class.

10. **MIMD** = **many instructions, many data**, describing **multicore and clusters**.

Memory Hierarchy and Types

1. The memory hierarchy has **4** main levels: **registers, cache, main memory, secondary storage**.
2. Moving **down** the hierarchy, access time **increases**, cost/bit **decreases**, capacity **increases**.
3. **Registers** are the **fastest and smallest**, sitting **inside the CPU**.
4. RAM is **2** types: **SRAM** and **DRAM**, both **volatile**.
5. **SRAM** uses **flip-flops**, needs **no refresh**, and is used in **cache**.
6. **DRAM** uses **capacitors**, needs **refresh**, and is used as **main memory**.
7. ROM is **non-volatile** and includes **PROM, EPROM, EEPROM, and Flash**.
8. **EPROM** is erased by **UV light**, while **EEPROM** is erased **electrically**.
9. **PROM** can be programmed **once**, whereas **EEPROM/Flash** are **rewritable**.
10. **Flash** memory is **block-erasable EEPROM**, used in **SSDs and USB drives**.

Cache Memory

1. Cache exploits **2** localities: **temporal (reuse soon)** and **spatial (nearby addresses)**.
2. There are **3** cache mapping techniques: **direct, associative, set-associative**.
3. **Direct mapping** sends a block to **one fixed line** using **block mod (#lines)**.
4. **Direct mapping** needs **no replacement policy** because each block has only one line.
5. **Associative mapping** lets a block go to **any line**, needing a **replacement policy**.
6. **Set-associative** mapping places a block in any line **within one set**, e.g., **k-way**.
7. A direct-mapped address has **3** fields: **Tag, Line, and Word/offset**.
8. A set-associative address replaces the **Line** field with a **Set** index.
9. Replacement policies number **4**: **LRU, FIFO, LFU, and Random**.
10. Write policies are **2**: **write-through** and **write-back**.
11. **Write-through** updates **cache and memory together**; **write-back** updates memory only on **eviction**.
12. **Write-back** uses a **dirty bit** to mark modified blocks; write-through does not.
13. Hit ratio **H** = hits / total accesses, so miss ratio is **1 – H**.
14. Average access time = **$H \cdot T_c + (1 - H) \cdot T_m$** , where **T_c** and **T_m** are cache and memory times.
15. A fully associative cache offers the **most flexibility** but needs the **most comparators**.

Virtual Memory and Management

1. **Paging** uses **fixed-size** pages and suffers **internal** fragmentation.
2. **Segmentation** uses **variable-size** segments and suffers **external** fragmentation.
3. A **page table** maps **virtual** → **physical** addresses.
4. The **TLB (Translation Lookaside Buffer)** is an **associative cache** for **page-table entries**.
5. **Memory interleaving** spreads consecutive addresses across **multiple banks** for parallel access.
6. **Associative (content-addressable) memory** is searched by **content**, not by **address**.
7. Virtual memory lets a program exceed **physical RAM** by paging to **disk**.
8. A TLB **hit** avoids a slow **page-table** lookup, speeding address translation.

Input-Output Organization

1. There are **3** data-transfer modes: **programmed I/O, interrupt-driven I/O, and DMA**.
2. **Programmed I/O** keeps the CPU **busy-waiting/polling**, making it the **slowest**.
3. **Interrupt-driven I/O** lets the device **interrupt** the CPU, avoiding busy waiting.
4. **DMA** transfers data **directly between device and memory**, bypassing the CPU — the **fastest** mode.
5. DMA has **2** modes: **cycle stealing (one word)** and **burst/block mode (whole block)**.
6. I/O addressing is **2** styles: **memory-mapped** and **isolated/I-O-mapped (IN/OUT)**.

7. Interrupts are classified as **maskable vs non-maskable**, **hardware vs software**, and **internal vs external**.
8. An **NMI (non-maskable interrupt) cannot be disabled/masked**.
9. **Daisy chaining** is a **hardware** priority scheme where the device **nearest the CPU** has **highest priority**.
10. **Polling** is a **software** priority method checking devices in a **fixed order**.
11. An **interrupt service routine (ISR)** runs when an interrupt is accepted, then control **returns**.
12. An **I/O Processor (IOP)** runs its own I/O programs, handling transfers **independently** of the CPU.
13. The I/O bus carries **3** signal groups: **address, data, and control** lines.
14. In DMA, the CPU only **initiates** the transfer and is notified at the **end**.